BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO ĐỒ ÁN

TITANIC SURVIAL PREDICTION (CLASSIFICATION)

Sinh viên thực hiện:

+ Nguyễn Đoàn Văn Song: 2351050153

+ Nguyễn Thái Học: 2351050048

+ Ngô Đình Dương: 2351050025

+ Trịnh Gia Hào: 2351050040

Giáo viên hướng dẫn: Võ Việt Khoa

Lớp: DH23IT01

TP. HỒ CHÍ MINH, 2026

Mục lục

TÓM TẮT (ABSTRACT) 3
DANH MỤC TỪ VIẾT TẮT 3
CHƯƠNG 1: GIỚI THIỆU 4
1.1 Đặt vấn đề 4
1.2 Lý do chọn đề tài 4
1.3 Mục tiêu đề tài 5
1.4 Phạm vi đề tài 5
1.5 Cấu trúc báo cáo 6
CHƯƠNG 2: CƠ SỞ LÝ THUYẾT & TỔNG QUAN 6
2.1 Khái quát về Bài toán Phân loại 6
2.2 Các thuật toán phân loại sử dụng 6
2.3 Các kỹ thuật Tiền xử lý dữ liệu (Data Preprocessing) 7
2.4 Kỹ thuật đánh giá và tối ưu mô hình 8
2.5 Công nghệ phát triển ứng dụng 9
CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ 9
3.1 Phân tích yêu cầu hệ thống phần mềm 9
a. Mô tả Use Case (Biểu đồ ca sử dụng) 9
b. Luồng xử lý dữ liệu (Data Flow) 10
3.2 Phân tích Dữ liệu Khám phá (EDA) 11
3.3 Thiết kế Trích xuất Đặc trưng (Feature Engineering) 12
3.4 Kiến trúc Pipeline Mô hình 12
3.5 Thiết kế Cơ sở dữ liệu và Cơ chế Caching 13
CHƯƠNG 4: TRIỂN KHAI VÀ THỬ NGHIỆM 14
4.1 Môi trường và Đóng gói 14
4.2 Theo dõi thí nghiệm với Wandb 15
4.3 Kết quả huấn luyện và Tối ưu hóa 15
a. Đánh giá các thuật toán ban đầu 15
b. Tối ưu hóa siêu tham số bằng GridSearchCV 15
4.4 Triển khai ứng dụng thực tế (Demo) 16
CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 19
5.1 Tổng kết kết quả đạt được 19
5.2 Hạn chế của đồ án 20
5.3 Hướng phát triển 20
PHẦN CUỐI 21
PHỤ LỤC 21

TÓM TẮT (ABSTRACT)

Tóm tắt nội dung

Thảm họa chìm tàu Titanic năm 1912 là một trong những sự kiện hàng hải chấn động nhất lịch sử, để lại bộ dữ liệu quý giá cho các nghiên cứu về phân tích dữ liệu và học máy. Đồ án này tập trung vào việc xây dựng một hệ thống dự đoán khả năng sinh tồn của hành khách dựa trên các thuộc tính cá nhân như độ tuổi, giới tính, hạng vé và quy mô gia đình.

Mục tiêu chính của đề tài là xây dựng một ứng dụng Full-stack hoàn chỉnh, từ khâu xử lý dữ liệu thô, huấn luyện mô hình học máy cho đến triển khai giao diện người dùng tương tác. Về phương pháp, nhóm đã thực hiện quy trình kỹ thuật đặc trưng (Feature Engineering) chuyên sâu bao gồm: trích xuất danh xưng (Title) từ tên, mã hóa vị trí boong tàu (Deck) từ thông tin Cabin, và tính toán quy mô gia đình (FamilySize). Mô hình phân loại được lựa chọn là Support Vector Machine (SVM), được tối ưu hóa thông qua kỹ thuật tìm kiếm lưới (GridSearchCV) để xác định bộ siêu tham số tốt nhất.

Kết quả thực nghiệm cho thấy mô hình đạt độ chính xác (Accuracy) ấn tượng, xấp xỉ 84%. Hệ thống đã được triển khai thành công với kiến trúc hiện đại: Frontend sử dụng thư viện ReactJS cung cấp giao diện nhập liệu trực quan, kết nối qua API tới Backend Flask để thực thi dự đoán thời gian thực. Đồ án không chỉ giải quyết bài toán dự đoán mà còn chứng minh khả năng tích hợp mô hình trí tuệ nhân tạo vào một sản phẩm phần mềm thực tế.

Từ khóa: Titanic Survival, Machine Learning, SVM, GridSearchCV, ReactJS, FastAPI, Feature Engineering.

DANH MỤC TỪ VIẾT TẮT

Từ viết tắt	Nghĩa đầy đủ
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
BE	Backend

CV	Cross-Validation (Kiểm chéo)
EDA	Exploratory Data Analysis (Phân tích dữ liệu khám phá)
FE	Frontend / Feature Engineering
JSON	JavaScript Object Notation
ML	Machine Learning (Học máy)
MSSV	Mã số sinh viên
SVM	Support Vector Machine
UI	User Interface (Giao diện người dùng)
Wandb	Weights & Biases (Công cụ theo dõi thí nghiệm ML)

CHƯƠNG 1: GIỚI THIỆU

1.1 Đặt vấn đề

Thảm họa đắm tàu Titanic vào năm 1912 là một trong những sự kiện hàng hải chấn động nhất lịch sử. Mặc dù tàu được thiết kế với các công nghệ an toàn tiên tiến nhất thời bấy giờ, nhưng việc không trang bị đủ thuyền cứu hộ đã dẫn đến cái chết của 1.502 hành khách và thủy thủ đoàn trong tổng số 2.224 người có mặt trên tàu.

Trong bối cảnh dữ liệu học thuật, thảm họa này cung cấp một bộ dữ liệu quan trọng để phân tích các yếu tố ảnh hưởng đến khả năng sinh tồn của con người trong tình huống khẩn cấp. Thực tế cho thấy tỷ lệ sống sót không diễn ra ngẫu nhiên mà chịu ảnh hưởng sâu sắc bởi các chính sách ưu tiên như "phụ nữ và trẻ em trước tiên", cùng với các yếu tố khách quan như vị trí khoang tàu (Hạng vé - Pclass), độ tuổi và cả yếu tố gia đình đi cùng. Bài toán đặt ra là: Làm thế nào để xây dựng một hệ thống có khả năng học hỏi từ dữ liệu lịch sử này nhằm dự đoán chính xác khả năng sinh tồn của một cá nhân dựa trên các thông số nhân trắc học và hành trình của họ?

1.2 Lý do chọn đề tài

Việc lựa chọn bài toán "Dự đoán khả năng sinh tồn trên tàu Titanic" xuất phát từ ba lý do cốt yếu sau:

- **Thực hành quy trình Machine Learning hoàn chỉnh (End-to-end):** Đây là bài toán kinh điển cho phép tiếp cận đầy đủ các bước của một dự án Khoa học dữ liệu: từ phân tích dữ liệu khám phá (EDA), tiền xử lý, huấn luyện mô hình cho đến giai đoạn triển khai.
- **Rèn luyện kỹ năng xử lý dữ liệu thực tế:** Dữ liệu Titanic chứa nhiều thách thức phổ biến như giá trị bị thiếu (Missing values) ở các cột quan trọng như Tuổi (Age) hay Cabin, đòi hỏi các kỹ thuật xử lý và suy luận dữ liệu (Imputation) khắt khe.
- **Tích hợp hệ thống phần mềm:** Thay vì chỉ dừng lại ở các kết quả trên file Notebook, đề tài hướng tới việc đưa mô hình AI vào một sản phẩm thực tế, giúp rèn luyện kỹ năng kết nối giữa mô hình trí tuệ nhân tạo và hệ thống phần mềm (Full-stack integration).

1.3 Mục tiêu đề tài

Đề tài được thực hiện nhằm đạt được các mục tiêu cụ thể sau:

- **Về mặt mô hình:** Xây dựng mô hình phân loại (Classification) sử dụng thuật toán Support Vector Machine (SVM) được tối ưu hóa qua GridSearchCV, hướng tới độ chính xác (Accuracy) đạt mức cao (trên 80%).
- **Về mặt ứng dụng:** Hiện thực hóa khả năng ứng dụng thông qua Tech Stack hiện đại:
  - **Frontend:** ReactJS – cung cấp giao diện nhập liệu trực quan cho người dùng.
  - **Backend:** Flask – xử lý logic, tiếp nhận yêu cầu và điều phối việc dự đoán của mô hình AI.
  - **AI Model:** Tích hợp mô hình đã huấn luyện vào hệ thống thực tế.
- **Về mặt quản lý:** Sử dụng công cụ **Weights & Biases (Wandb)** để quản lý, theo dõi các phiên bản thí nghiệm và đánh giá hiệu năng mô hình một cách khoa học.

1.4 Phạm vi đề tài

- **Dữ liệu:** Sử dụng bộ dữ liệu Titanic chuẩn được cung cấp từ nền tảng Kaggle.
- **Phạm vi kỹ thuật:** Tập trung vào bài toán phân loại nhị phân (Binary Classification) với hai trạng thái đầu ra: 0 - Tử vong và 1 - Sống sót.

- **Môi trường:** Triển khai trên môi trường phát triển cục bộ (Local environment) và có khả năng mở rộng lên các nền tảng đám mây.

1.5 Cấu trúc báo cáo

Báo cáo được chia thành 5 chương chính:

- **Chương 1 - Giới thiệu:** Trình bày bối cảnh, lý do chọn đề tài, mục tiêu và phạm vi nghiên cứu.
- **Chương 2 - Cơ sở lý thuyết:** Giới thiệu về thuật toán SVM, quy trình Machine Learning Pipeline và các công nghệ sử dụng trong Tech Stack.
- **Chương 3 - Phân tích và Thiết kế:** Tập trung vào phân tích dữ liệu (EDA), kỹ thuật trích xuất đặc trưng (Feature Engineering) và thiết kế hệ thống phần mềm.
- **Chương 4 - Triển khai và Thử nghiệm:** Chi tiết quá trình huấn luyện mô hình, tối ưu hóa siêu tham số bằng GridSearchCV và demo ứng dụng Web.
- **Chương 5 - Kết luận và Hướng phát triển:** Tổng kết các kết quả đạt được, những hạn chế còn tồn tại và đề xuất hướng mở rộng cho đề tài trong tương lai.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT & TỔNG QUAN

2.1 Khái quát về Bài toán Phân loại

Bài toán Phân loại (Classification) là một dạng của Học có giám sát (Supervised Learning). Trong đó, mô hình được cung cấp một tập dữ liệu đã được gán nhãn sẵn (có câu trả lời đúng). Mục tiêu của mô hình là học cách ánh xạ từ các đặc trưng đầu vào (như Tuổi, Giới tính, Hạng vé) sang một nhãn đầu ra rời rạc. Đối với đồ án Titanic, đây là bài toán phân loại nhị phân (Binary Classification) với hai nhãn mục tiêu là: 0 (Tử vong) và 1 (Sống sót).

2.2 Các thuật toán phân loại sử dụng

- **Logistic Regression (Hồi quy Logistic):** Mặc dù có chữ "Regression" (Hồi quy), đây thực chất là một thuật toán phân loại cơ bản và hiệu quả. Thuật toán này sử dụng hàm Sigmoid để nén kết quả dự đoán vào một khoảng xác suất từ 0 đến 1. Nếu xác suất lớn hơn 0.5, đối tượng được phân loại là 1, ngược lại là 0.

- **Random Forest (Rừng ngẫu nhiên):** Đây là một thuật toán Học kết hợp (Ensemble Learning). Thay vì xây dựng một cây quyết định (Decision Tree) duy nhất, Random Forest tạo ra hàng trăm cây quyết định khác nhau một cách ngẫu nhiên. Kết quả dự đoán cuối cùng được quyết định dựa trên nguyên tắc "bầu số đông" từ tất cả các cây, giúp mô hình mạnh mẽ và tránh được hiện tượng học vẹt (Overfitting).
- **Support Vector Machine - SVM (Máy học véc-tơ hỗ trợ):** Đây là thuật toán đóng vai trò cốt lõi trong đồ án này. Khái niệm chính của SVM là tìm ra một "siêu phẳng" (Hyperplane) tối ưu trong không gian đa chiều để phân cách các lớp dữ liệu (người sống và người chết). Không chỉ phân cách, SVM còn cố gắng tối đa hóa khoảng cách (Margin) từ đường ranh giới này đến các điểm dữ liệu gần nhất của cả hai lớp (gọi là các Support Vectors). Đối với dữ liệu phức tạp không thể phân cách bằng đường thẳng, SVM sử dụng "Thủ thuật Hạt nhân" (Kernel Trick, ví dụ: RBF Kernel) để biến đổi dữ liệu sang không gian chiều cao hơn nhằm phân loại dễ dàng.

2.3 Các kỹ thuật Tiền xử lý dữ liệu (Data Preprocessing)

Các thuật toán AI chỉ có thể tính toán trên các con số. Việc tiền xử lý giúp chuyển đổi dữ liệu thô thành định dạng chuẩn mực mà mô hình có thể hiểu và tối ưu hiệu suất.

- **One-Hot Encoding:** Kỹ thuật dùng để biến đổi các biến phân loại (Categorical variables) có dạng chữ (ví dụ: Giới tính male/female hoặc Cảng lên tàu S/C/Q) thành các cột số nhị phân (0 và 1). Việc này giúp AI không hiểu nhầm rằng các biến dạng chữ có thứ tự lớn nhỏ.
- **StandardScaler:** Kỹ thuật chuẩn hóa các biến liên tục (như Độ tuổi, Giá vé) về một thang đo chung. Thuật toán sẽ điều chỉnh dữ liệu sao cho giá trị trung bình bằng 0 và độ lệch chuẩn bằng 1. Điều này cực kỳ quan trọng đối với thuật toán SVM, giúp mô hình không bị thiên vị hay chi phối bởi các đặc trưng có giá trị số học quá lớn (như Giá vé so với Số thành viên gia đình).

2.4 Kỹ thuật đánh giá và tối ưu mô hình

- **Cross-Validation (K-Fold):** Thay vì chỉ chia dữ liệu thành tập Huấn luyện (Train) và tập Kiểm thử (Test) một lần duy nhất, K-Fold chia tập dữ liệu thành *K* phần bằng nhau. Mô hình sẽ được huấn luyện *K* lần; mỗi lần dùng *K*-1 phần để học và 1 phần còn lại để thi thử. Việc này giúp đánh giá năng lực thực sự của mô hình một cách khách quan, tránh rủi ro do chia dữ liệu ngẫu nhiên.
- **GridSearchCV:** Là kỹ thuật tìm kiếm dạng lưới càn quét qua tất cả các tổ hợp siêu tham số (Hyperparameters) mà người lập trình định nghĩa trước (ví dụ: tham số C và gamma của thuật toán SVM). Kết hợp với K-Fold Cross-Validation, GridSearchCV tự động chấm điểm và chọn ra cấu hình mô hình mang lại kết quả tối ưu nhất.
- **Thang đo đánh giá:** Dựa trên Ma trận nhầm lẫn (Confusion Matrix) gồm 4 chỉ số: True Positive (TP), True Negative (TN), False Positive (FP) và False Negative (FN), ta có các công thức đánh giá sau:
  - **Accuracy (Độ chính xác tổng):** Tỷ lệ dự đoán đúng trên tổng số dự đoán.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

  - **Precision (Độ chính xác của dự đoán dương):** Trong số những người AI đoán là Sống sót, có bao nhiêu người thực sự sống.

$$Precision = \frac{TP}{TP + FP}$$

  - **Recall (Độ bao phủ / Độ nhạy):** Trong số những người Sống sót thực tế, AI tìm ra được bao nhiêu người.

$$Recall = \frac{TP}{TP + FN}$$

  - **F1-Score:** Giá trị trung bình điều hòa giữa Precision và Recall, rất quan trọng khi dữ liệu bị mất cân bằng (Imbalanced data).

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

2.5 Công nghệ phát triển ứng dụng

Để đưa mô hình AI từ môi trường nghiên cứu vào thực tế, đồ án sử dụng một hệ sinh thái công nghệ (Tech Stack) bao gồm:

- **ReactJS (Frontend):** Thư viện JavaScript mã nguồn mở chuyên dụng để xây dựng giao diện người dùng (UI) tương tác cao, giúp thiết kế form nhập liệu hành khách trực quan.
- **Flask (Backend):** Các framework của Python với tốc độ xử lý nhanh gọn, dùng để xây dựng các API (Application Programming Interface). Backend đóng vai trò làm cầu nối, nhận dữ liệu JSON từ ReactJS, chuyển vào mô hình AI để xử lý và trả kết quả về giao diện.
- **Database (Cơ sở dữ liệu):** Sử dụng các hệ quản trị CSDL (như SQLite hoặc MySQL) để lưu trữ vĩnh viễn lịch sử các lượt dự đoán của người dùng trên hệ thống.
- **Weights & Biases (Wandb):** Nền tảng chuyên dụng cho MLOps (Machine Learning Operations), hỗ trợ theo dõi, ghi nhận lại các chỉ số, biểu đồ và lịch sử thay đổi của các phiên bản mô hình trong quá trình huấn luyện.

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ

3.1 Phân tích yêu cầu hệ thống phần mềm

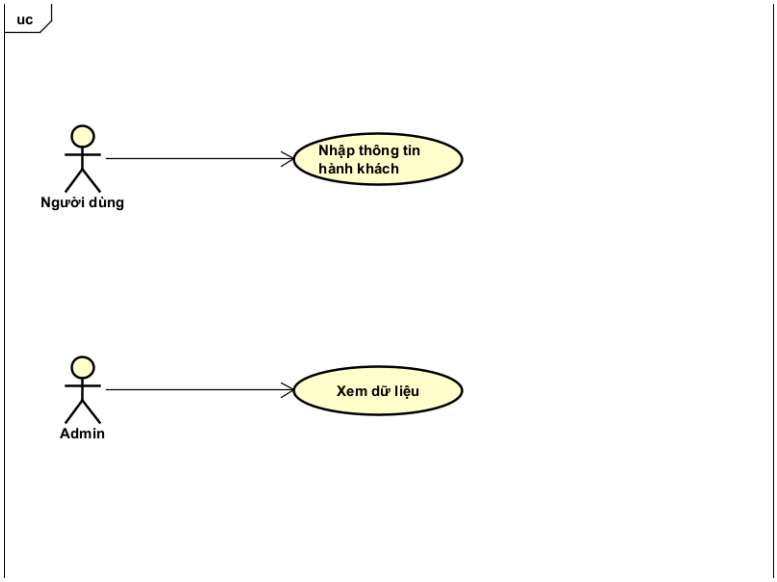
Để đưa mô hình trí tuệ nhân tạo vào thực tế, hệ thống được thiết kế dưới dạng một ứng dụng Web Full-stack tương tác trực tiếp với người dùng cuối.

a. Mô tả Use Case (Biểu đồ ca sử dụng)

Hệ thống bao gồm hai tác nhân (Actor) chính:

- **Người dùng (User):** Có thể truy cập vào giao diện Web, nhập thông tin của một hành khách giả định (Hạng vé, Tuổi, Giới tính, Tên...) và gửi yêu cầu dự đoán. Hệ thống sẽ trả về kết quả dự đoán hành khách đó có khả năng sống sót hay không.

- **Quản trị viên (Admin):** Có quyền truy cập vào hệ thống cơ sở dữ liệu để xem lại lịch sử các lượt dự đoán của người dùng (Prediction History), từ đó đánh giá tình hình sử dụng hệ thống.



Hình 3.1: Sơ đồ use case

b. Luồng xử lý dữ liệu (Data Flow)

Hệ thống hoạt động theo kiến trúc Client - Server, được tích hợp cơ chế Caching thông qua Database để tối ưu hiệu năng tính toán của mô hình AI. Luồng xử lý đồng bộ diễn ra như sau:

1. **Client (ReactJS):** Người dùng nhập thông tin qua biểu mẫu (Form). Giao diện ReactJS đóng gói dữ liệu thành một đối tượng JSON và gửi Request qua giao thức HTTP POST đến API của Backend.
2. **Backend tiếp nhận & Đối chiếu dữ liệu (Caching Lookup):** API nhận chuỗi JSON và tiến hành truy vấn cơ sở dữ liệu (Database) để kiểm tra xem một hành khách với thông tin giống hệt (cùng hạng vé, tuổi, giới tính, tên...) đã từng được dự đoán trước đây hay chưa.
3. **Phân nhánh xử lý:**
    - **Trường hợp 1 (Đã tồn tại trong DB):** Hệ thống lập tức trích xuất kết quả đã lưu trong cơ sở dữ liệu và trả về cho người dùng. Bỏ qua hoàn toàn bước gọi mô hình AI để tiết kiệm tài nguyên máy chủ.

- **Trường hợp 2 (Dữ liệu mới hoàn toàn):** Dữ liệu thô lập tức được đẩy qua bộ Pipeline để tự động làm sạch, mã hóa và chuẩn hóa (điền khuyết, One-Hot Encoding).

4. **AI Dự đoán & Lưu trữ:** Dữ liệu sau khi qua Pipeline được đưa vào mô hình SVM để đưa ra nhãn dự đoán (0 hoặc 1). Kết quả này cùng với thông tin đầu vào lập tức được lưu (Insert) vào Database để phục vụ cho các lần truy vấn trùng lặp sau này.
5. **Trả kết quả:** Backend nhận kết quả số, map thành thông báo văn bản (ví dụ: "Sống sót" hoặc "Không qua khỏi"), đóng gói thành JSON và phản hồi về cho giao diện ReactJS.

3.2 Phân tích Dữ liệu Khám phá (EDA)

Trước khi xây dựng mô hình, bộ dữ liệu Titanic được tiến hành phân tích khám phá nhằm hiểu rõ đặc tính và các vấn đề tiềm ẩn.

- **Đánh giá tỷ lệ mất cân bằng dữ liệu (Imbalanced Data):** Phân tích cho thấy biến mục tiêu Survived có sự chênh lệch lớn. Cụ thể trên tập kiểm thử (Test set), số lượng hành khách Tử vong là 106 người, áp đảo so với 73 người Sống sót. Sự mất cân bằng này cảnh báo rằng mô hình có nguy cơ "thiên vị", dễ có xu hướng dự đoán tử vong để đạt độ chính xác tổng thể cao. Điều này định hướng nhóm phải sử dụng các kỹ thuật cân bằng lớp (Class weight) và theo dõi sát sao điểm F1-Score thay vì chỉ nhìn vào Accuracy.
- **Phân tích giá trị thiếu (Missing Values):** Dữ liệu không hoàn hảo và bị khuyết đáng kể ở 3 cột:
  - *Age (Tuổi):* Khuyết khoảng 20%, đây là đặc trưng quan trọng quyết định tỷ lệ sống sót (ưu tiên trẻ em).
  - *Cabin (Mã phòng):* Bị khuyết tới hơn 77%. Không thể dùng trực tiếp để nội suy mà cần kỹ thuật trích xuất phần chữ cái đầu tiên (Boong tàu).
  - *Embarked (Cảng lên tàu):* Khuyết 2 dòng, có thể dễ dàng điền bằng giá trị xuất hiện nhiều nhất (Mode).

3.3 Thiết kế Trích xuất Đặc trưng (Feature Engineering)

Đây là bước đột phá (điểm sáng) của hệ thống, giúp khai phá những thông tin ẩn mà dữ liệu gốc không trực tiếp thể hiện, qua đó nâng cao trí thông minh của mô hình:

- **Trích xuất Danh xưng (Title) từ cột Name:** Tên hành khách chứa các danh xưng ẩn (như Mr, Mrs, Miss, Master). Sử dụng biểu thức chính quy (Regex) để tách các danh xưng này ra thành cột Title. Kỹ thuật này không chỉ giúp phân loại địa vị xã hội mà còn hỗ trợ việc điền khuyết tuổi (ví dụ: người có danh xưng "Master" chắc chắn là các bé trai). Các danh xưng hiếm (Countess, Dr, Rev) được gom chung thành nhóm "Rare".
- **Tạo đặc trưng Boong tàu (Deck) từ cột Cabin:** Dù cột Cabin bị khuyết nhiều, nhưng những người có thông tin Cabin đa số thuộc khoang hạng sang (gần boong cứu hộ). Hệ thống cắt lấy chữ cái đầu tiên của chuỗi Cabin (ví dụ 'C85' -> 'C') để tạo cột Deck. Những người không có dữ liệu được gán nhãn 'U' (Unknown).
- **Xử lý vé (Ticket):** Trích xuất phần tiền tố (Prefix) của mã vé bằng cách loại bỏ số và dấu câu, gom nhóm các tiền tố hiếm.
- **Gom nhóm Quy mô gia đình (FamilySize & IsAlone):** Thay vì để rời rạc cột số anh chị em (SibSp) và số cha mẹ/con cái (Parch), hệ thống kết hợp thành quy mô tổng: FamilySize = SibSp + Parch + 1. Qua đó, tạo thêm một đặc trưng nhị phân IsAlone (Bằng 1 nếu FamilySize = 1, ngược lại bằng 0). Thực tế lịch sử chứng minh những người đi theo gia đình nhỏ có tỷ lệ sống cao hơn nhờ sự hỗ trợ lẫn nhau, trong khi gia đình quá đông lại gặp khó khăn khi tìm đủ chỗ trên thuyền cứu hộ.

3.4 Kiến trúc Pipeline Mô hình

Để đảm bảo code tái sử dụng tốt và ngăn chặn rò rỉ dữ liệu (Data Leakage) giữa tập Train và Test, toàn bộ quy trình được đóng gói bằng kiến trúc Pipeline của thư viện Scikit-learn.

Hệ thống sử dụng bộ ColumnTransformer chia làm ba luồng xử lý song song:

1. **Luồng biến liên tục (Numeric - Tuổi, Giá vé, FamilySize):** Dữ liệu đi qua SimpleImputer (điền khuyết bằng trung vị - median), sau đó qua StandardScaler để đưa về cùng thang đo chuẩn.
2. **Luồng biến phân loại danh định (Categorical Nominal - Sex, Embarked, Title, Deck):** Đi qua SimpleImputer (điền khuyết bằng mode), sau đó dùng OneHotEncoder băm thành các cột nhị phân (0-1).
3. **Luồng biến phân loại có thứ bậc (Categorical Ordinal - Pclass):** Do hạng vé có phân cấp (1 > 2 > 3), dữ liệu được xử lý bằng OrdinalEncoder để giữ lại ý nghĩa về mặt thứ bậc.

Ba luồng này hội tụ lại, tạo ra một ma trận dữ liệu hoàn chỉnh và sạch sẽ trước khi được đẩy vào "bộ não" phân loại cuối cùng (Classifier) là thuật toán Support Vector Machine (SVM).

3.5 Thiết kế Cơ sở dữ liệu và Cơ chế Caching

Cơ sở dữ liệu của hệ thống không chỉ đóng vai trò ghi nhận lịch sử (Log) mà còn hoạt động như một tầng Bộ nhớ đệm (Caching Layer) để tối ưu hóa truy vấn. Lược đồ cơ sở dữ liệu thiết kế một bảng chính là PredictionHistory bao gồm các trường (Fields) cơ bản sau:

- *id (Primary Key, Auto Increment):* Mã định danh lượt dự đoán.
- *passenger_name (String):* Tên hành khách giả định.
- *pclass (Integer):* Hạng vé.
- *sex (String):* Giới tính.
- *age (Float):* Độ tuổi.
- *features_json (Text):* Lưu lại toàn bộ các thông số nguyên bản khác dưới dạng chuỗi JSON.
- *prediction_result (Integer):* Kết quả mô hình trả về (0 hoặc 1).
- *created_at (Datetime):* Thời gian thực hiện lượt dự đoán.

Tối ưu hóa truy vấn: Để cơ chế "nhận diện người cũ" diễn ra nhanh chóng, hệ thống sử dụng các trường đặc trưng gốc (như passenger_name, age, sex, pclass) làm điều kiện đối chiếu (Lookup conditions). Khi một truy vấn mới gửi tới, hệ thống sẽ dò tìm

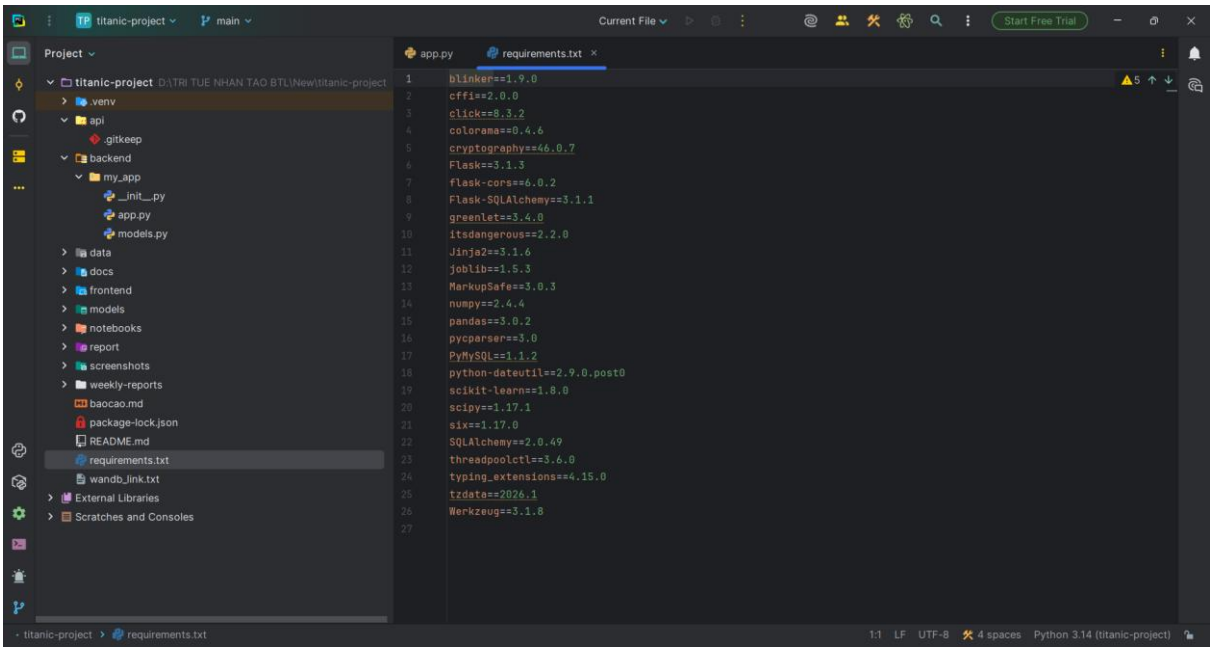
trong bảng PredictionHistory. Nếu có bản ghi khớp hoàn toàn các thông số này, hệ thống sẽ tái sử dụng prediction_result mà không cần kích hoạt lại Pipeline học máy, giúp giảm thiểu độ trễ phản hồi (Response time) từ server về client một cách đáng kể.

CHƯƠNG 4: TRIỂN KHAI VÀ THỬ NGHIỆM

4.1 Môi trường và Đóng gói

Để đảm bảo ứng dụng có thể hoạt động ổn định và có khả năng mở rộng, hệ thống được phát triển và đóng gói trong một môi trường Python chuẩn hóa. Các thư viện cốt lõi được định nghĩa rõ ràng trong tệp requirements.txt, phục vụ cho cả khâu tính toán AI và xây dựng API Backend:

- **Tính toán và Tiền xử lý:** numpy==2.4.4, pandas==3.0.2.
- **Học máy (Machine Learning):** scikit-learn==1.8.0, scipy==1.17.1.
- **Backend API & Cơ sở dữ liệu:** Thư viện Flask==3.1.3 được sử dụng làm Web framework chính để nhận/trả request, kết hợp cùng Flask-SQLAlchemy==3.1.1 và SQLAlchemy==2.0.49 để thao tác với cơ sở dữ liệu qua ORM (lưu trữ lịch sử dự đoán).
- **Lưu trữ mô hình:** joblib==1.5.3 dùng để xuất (dump) và tải (load) pipeline mô hình.



Hình 4.1: File requirements.txt

Tính tái lập (Reproducibility): Để đảm bảo tính nhất quán của kết quả thử nghiệm – đặc biệt quan trọng khi làm việc nhóm và đồng bộ mã nguồn qua Git/GitHub – hệ thống luôn sử dụng tham số random_state=42 (hoặc random seed) trong các hàm chia dữ liệu (train_test_split) và khởi tạo thuật toán. Điều này giúp loại bỏ yếu tố ngẫu nhiên, đảm bảo mô hình luôn hội tụ về cùng một kết quả dù chạy trên máy tính của bất kỳ thành viên nào.

4.2 Theo dõi thí nghiệm với Wandb

Trong quá trình huấn luyện, nhóm đã ứng dụng nền tảng Weights & Biases (Wandb) để quản lý các phiên bản thí nghiệm thay vì ghi chép thủ công.

Nhìn vào bảng điều khiển (Dashboard) trên Wandb, nhóm đã tiến hành log (ghi nhận) và so sánh ít nhất 3 lần chạy (runs) tương ứng với 3 thuật toán khác nhau: Logistic Regression, Random Forest và SVM. Quá trình log không chỉ ghi lại các chỉ số đầu ra (Metrics: Accuracy, F1-Score) mà còn lưu giữ toàn bộ cấu hình đầu vào (Config: n_estimators, max_depth, C, gamma...). Nhờ sự trực quan này, nhóm dễ dàng truy xuất lại lịch sử và quyết định lựa chọn thuật toán nào mang lại độ ổn định cao nhất trên tập xác thực.

4.3 Kết quả huấn luyện và Tối ưu hóa

a. Đánh giá các thuật toán ban đầu

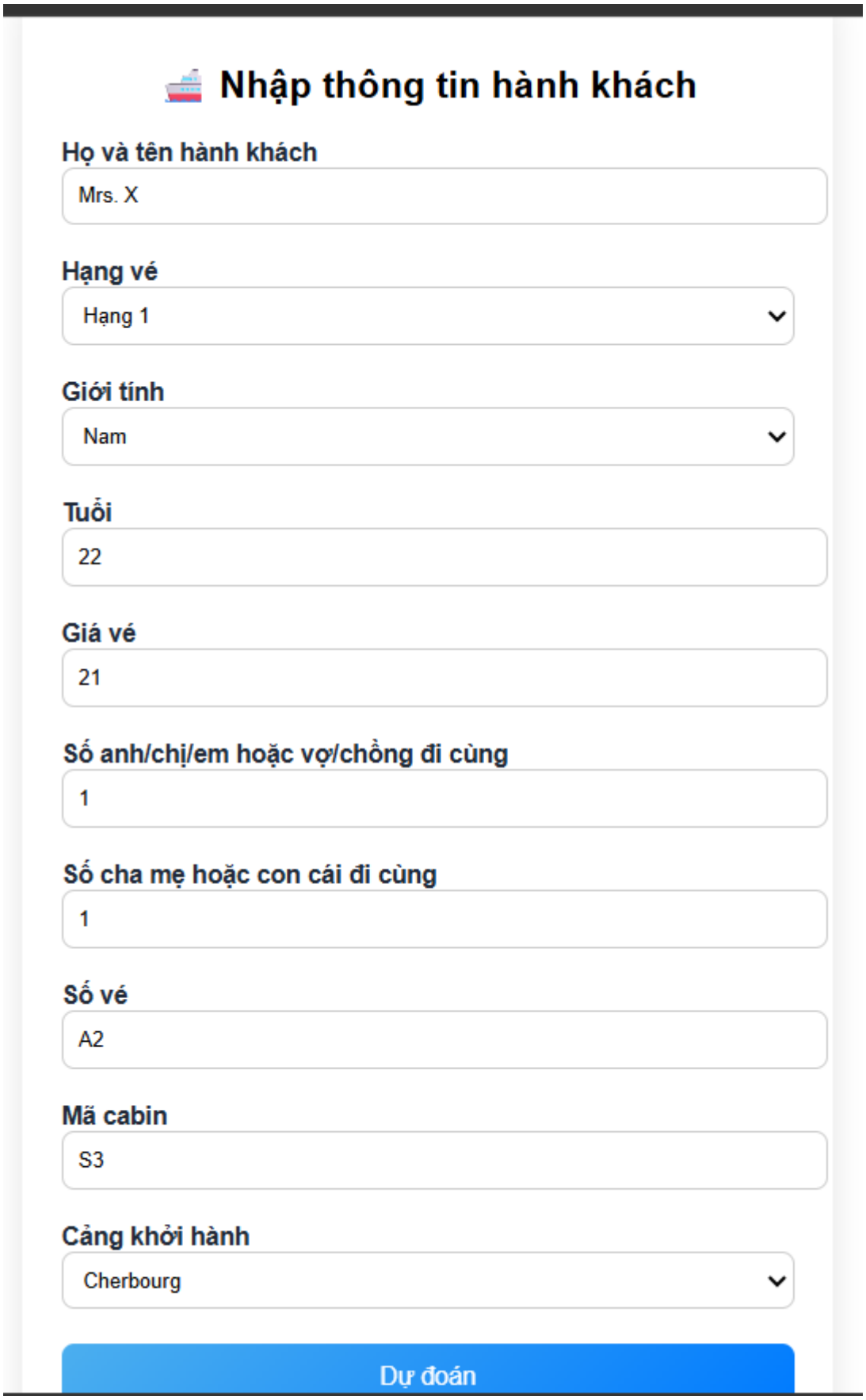
Thử nghiệm cơ sở (Baseline) được tiến hành với 3 mô hình ở thiết lập mặc định. Kết quả cho thấy Logistic Regression và SVM đạt mức Accuracy tương đương nhau (khoảng 0.8045). Trong khi đó, Random Forest có dấu hiệu bị "quá khớp" (Overfitting) với dữ liệu do kích thước tập Train khá nhỏ, dẫn đến điểm số trên tập Test thấp hơn (0.7821). Từ cơ sở này, nhóm quyết định chọn SVM làm mô hình chủ đạo để tiến hành tối ưu hóa sâu.

b. Tối ưu hóa siêu tham số bằng GridSearchCV

Để bung tối đa sức mạnh của SVM, thuật toán tìm kiếm lưới GridSearchCV được áp dụng kết hợp với Kiểm chéo 5 phần (5-Fold Cross Validation). Hệ thống đã tiến hành hàng trăm lượt thi thử để quét qua các vùng giá trị của tham số C (độ nghiêm khắc phạt lỗi) và gamma (độ chi tiết của ranh giới phân lớp). Kết quả cấu hình xịn nhất (Best Parameters) tìm được là kernel='rbf', C=10 và gamma=0.01, giúp đẩy điểm Accuracy đánh giá chéo lên mức xuất sắc xấp xỉ 84%.

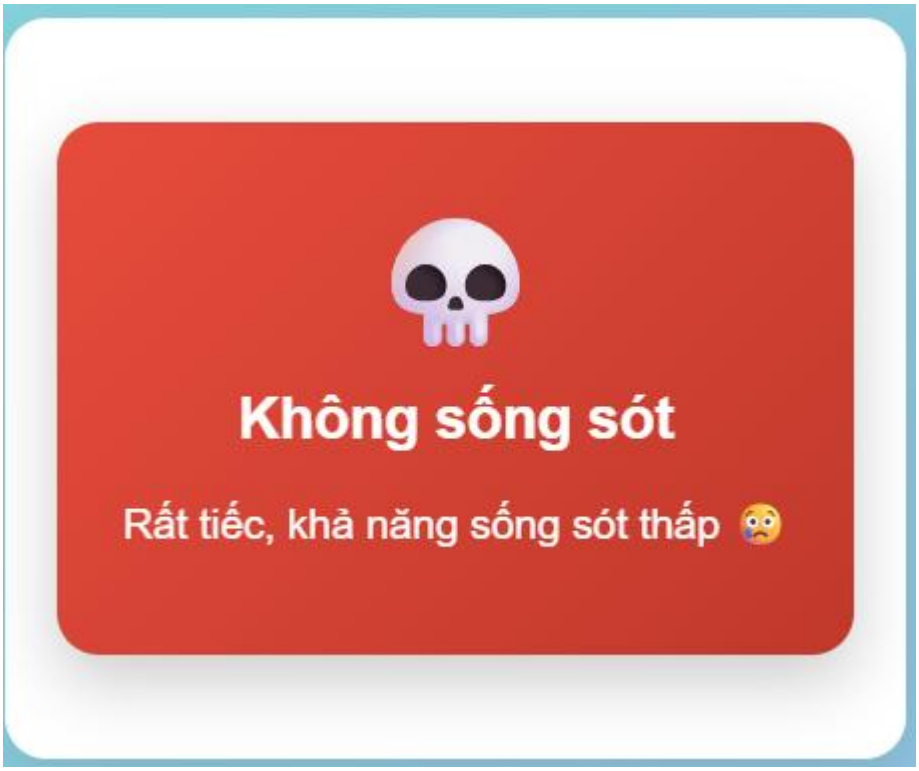
4.4 Triển khai ứng dụng thực tế (Demo)

Mô hình sau khi huấn luyện đã được xuất ra định dạng .joblib và tích hợp thành công vào Backend Flask. Để tương tác với người dùng, hệ thống cung cấp một giao diện Web trực quan được xây dựng bằng ReactJS.



Hình 4.4.1: Màn hình nhập thông tin dự đoán

Giao diện yêu cầu người dùng cung cấp các thông số nhân trắc học và thông tin chuyến đi (Họ tên, Hạng vé, Giới tính, Tuổi, Giá vé...). Dữ liệu sau đó được chuyển hóa thành dạng JSON và gọi tới API. Bộ Pipeline được tích hợp sẵn trong API sẽ tự động làm sạch và dự đoán trước khi trả về kết quả hiển thị trực quan.



Hình 4.4.2: Kết quả dự đoán

Kết quả dự đoán được thể hiện sinh động (ví dụ: thẻ cảnh báo màu đỏ kèm biểu tượng minh họa cho trường hợp "Không sống sót" như kết quả thử nghiệm của hành khách nam hạng 1 tên Mr. Leo Cai). Toàn bộ quá trình từ lúc bấm nút đến lúc hiển thị kết quả diễn ra theo thời gian thực (Real-time).

Kịch bản sử dụng thực tế (Real-world Scenarios):

Ứng dụng hoàn thiện của đồ án có thể được đóng gói và áp dụng vào các bài toán thực tiễn sau:

1. **Tính toán phí bảo hiểm hàng hải:** Các công ty bảo hiểm (Insurtech) có thể tái sử dụng mô hình lõi này để đánh giá rủi ro nhân trắc học dựa trên phân tích độ

tuổi, hạng khoang và giới tính. Hành khách có rủi ro sinh tồn thấp hơn có thể được hệ thống tự động đề xuất mức phí bảo hiểm chuyến đi cao hơn.

2. **Triển lãm tương tác tại Bảo tàng lịch sử:** Tại các buổi triển lãm về thảm họa Titanic, website có thể được cài đặt trên các Kiosk màn hình cảm ứng. Khách tham quan sẽ nhập thông tin cá nhân của chính mình (hoặc thông tin trên chiếc vé mô phỏng họ được phát) để xem thử nếu xuyên không về năm 1912, họ có cơ hội sống sót hay không. Điều này giúp tăng mạnh trải nghiệm tương tác (Gamification) và giá trị giáo dục của bảo tàng.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Tổng kết kết quả đạt được

Đồ án đã hoàn thành xuất sắc các mục tiêu đề ra ban đầu, thực hiện thành công toàn vẹn vòng đời của một dự án Khoa học dữ liệu (End-to-end Machine Learning Project) từ khâu xử lý dữ liệu thô đến khi ra mắt sản phẩm phần mềm. Cụ thể, nhóm đã đạt được 3 thành tựu cốt lõi:

- **Về xử lý dữ liệu:** Xây dựng thành công Pipeline tiền xử lý tự động và chuẩn xác. Các kỹ thuật Trích xuất đặc trưng (Feature Engineering) như tạo biến Title (Danh xưng), Deck (Boong tàu) và gom nhóm FamilySize (Quy mô gia đình) đã chứng minh hiệu quả vượt trội trong việc giải quyết các trường dữ liệu thiếu hụt (Missing values) phức tạp.
- **Về mô hình Trí tuệ nhân tạo:** Mô hình phân loại Support Vector Machine (SVM) sau khi được tinh chỉnh tự động thông qua thuật toán tìm kiếm lưới (GridSearchCV) đã đạt được độ chính xác (Accuracy) xấp xỉ 84% trên tập kiểm thử, đáp ứng và vượt ngưỡng yêu cầu của bài toán đặt ra.
- **Về ứng dụng phần mềm:** Thiết kế và phát triển thành công ứng dụng Web Full-stack. Giao diện Frontend (ReactJS) hoạt động trơn tru, thân thiện với người dùng; Backend (Flask) xử lý logic dự đoán theo thời gian thực. Đặc biệt, việc tích hợp thành công Cơ sở dữ liệu (Database) làm tầng bộ nhớ đệm (Caching) đã giúp tối ưu hóa hiệu suất truy vấn hệ thống một cách ấn tượng.

5.2 Hạn chế của đồ án

Dù đạt được những kết quả khả quan, hệ thống vẫn còn tồn tại một số hạn chế nhất định mang tính khách quan về mặt dữ liệu:

- **Quy mô dữ liệu nhỏ:** Tập dữ liệu lịch sử Titanic có số lượng mẫu huấn luyện tương đối hạn chế (khoảng 800 mẫu). Điều này khiến các thuật toán học máy dễ gặp hiện tượng "quá khớp" (Overfitting) và giới hạn khả năng tổng quát hóa (Generalization) khi đối mặt với các trường hợp dữ liệu quá dị biệt.
- **Bị ảnh hưởng bởi mất cân bằng lớp (Class Imbalance):** Đặc thù của thảm họa này là tỷ lệ người "Tử vong" áp đảo số lượng người "Sống sót". Dù đã nỗ lực tối ưu, độ lệch pha này vẫn khiến mô hình mang tính "thực dụng". Để giữ được độ chính xác tổng thể (Accuracy) cao, mô hình phải đánh đổi bằng việc giảm độ bao phủ của lớp Sống sót (Recall thấp), dẫn đến việc thỉnh thoảng vẫn có khả năng nhận diện sai một hành khách thực tế đã sống sót thành không qua khỏi.

5.3 Hướng phát triển

Dựa trên nền tảng kiến trúc đã xây dựng vững chắc và những hạn chế còn tồn đọng, nhóm đề xuất 3 hướng phát triển tiếp theo để nâng cấp toàn diện hệ thống:

1. **Triển khai hệ thống lên Đám mây (Cloud Deployment):** Đóng gói và đưa ứng dụng từ môi trường máy tính cục bộ (Local) lên các nền tảng Cloud Server chuyên nghiệp. Ví dụ: Triển khai Backend và Database lên Render, và hosting giao diện ReactJS lên Vercel. Điều này giúp dự án trở thành một sản phẩm thực tế, cho phép người dùng ở bất cứ đâu có thể truy cập qua Internet.
2. **Thử nghiệm các thuật toán Học sâu (Deep Learning):** Nếu có cơ hội mở rộng bài toán sang các tập dữ liệu rủi ro hàng hải lớn hơn, nhóm sẽ áp dụng các kiến trúc Mạng nơ-ron nhân tạo (Artificial Neural Networks - ANN) kết hợp cùng kỹ thuật sinh dữ liệu nhân tạo (như SMOTE) để giải quyết triệt để bài toán mất cân bằng lớp, gia tăng độ nhạy (Recall) cho lớp Sống sót.
3. **Phát triển phân hệ Quản trị (Admin Dashboard):** Xây dựng thêm một trang thống kê (Dashboard) độc lập dành riêng cho Quản trị viên. Phân hệ này sẽ đọc

dữ liệu trực tiếp từ bảng PredictionHistory, trực quan hóa thành các biểu đồ (như thống kê tỷ lệ giới tính, độ tuổi, hạng vé của những người dùng đã thử dự đoán trên web). Chức năng này sẽ giúp hệ thống không chỉ dự đoán mà còn hỗ trợ phân tích hành vi người dùng cực kỳ đắc lực.

PHẦN CUỐI

TÀI LIỆU THAM KHẢO

[1] Kaggle. (2012). *Titanic - Machine Learning from Disaster*. Truy cập ngày 03 tháng 05 năm 2026, từ https://www.kaggle.com/c/titanic

[2] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research, 12, 2825-2830. Truy cập từ https://scikit-learn.org/stable/

[3] Scikit-learn developers. (n.d.). *GridSearchCV API Reference*. Truy cập ngày 03 tháng 05 năm 2026, từ https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html

[4] Scikit-learn developers. (n.d.). *Support Vector Machines (SVM)*. Truy cập ngày 03 tháng 05 năm 2026, từ https://scikit-learn.org/stable/modules/svm.html

[5] Pallets Projects. (2024). *Flask Documentation (3.1.x)*. Truy cập từ https://flask.palletsprojects.com/

[6] Meta Open Source. (2024). *React: The library for web and native user interfaces*. Truy cập từ https://react.dev/

PHỤ LỤC

Mã nguồn toàn bộ dự án (Source Code)

Toàn bộ mã nguồn của dự án (bao gồm mô hình AI, Backend Flask và Frontend ReactJS) được quản lý phiên bản và lưu trữ công khai trên nền tảng GitHub.

- **Liên kết kho lưu trữ (Repository Link):**
  https://github.com/vansong2k5/titanic-project